

# RespiraApp

## Plataforma Integral de Gestión de Bienestar

*Estudio de Yoga "Respira Profundo"*

---

### INFORME EJECUTIVO DE DESARROLLO

Sesión: Junio 2026 · Elaborado por: Yuliana Choque - Yulianna Luna - Valentina Zamora - Fabiana Zepeda

## 1. Contexto del Proyecto

---

El estudio de yoga "Respira Profundo", dirigido por Valentina Reyes, opera con 4 instructores y una oferta de 30 clases semanales. Previo a este proyecto, toda la gestión se realizaba de forma manual: reservas via mensajes directos en Instagram, control de asistencia en hojas Excel y descuento manual de clases de los paquetes de alumnos.

### 1.1 Diagnóstico AS IS (Situación Previa)

- Gestión de reservas 100% manual vía Instagram — dependiente de respuesta humana
- Control de asistencias en planillas Excel locales, propensas a errores y pérdida de datos
- Descuento manual de clases en paquetes (10, 20 o ilimitado) con alta tasa de error
- Cero visibilidad analítica: sin datos consolidados de rentabilidad, demanda o fidelización
- Sin respaldo ni auditoría de registros históricos

#### Ineficiencias identificadas (Pain Points)

- Valentina: "Ceguera administrativa" — incapacidad de tomar decisiones basadas en datos
- Alumnos: Frustración por cobros indebidos y falta de autonomía para consultar saldo
- Instructores: Sin herramientas digitales de registro de asistencia

### 1.2 Objetivo del Proyecto (TO BE)

Diseñar, desarrollar y entregar una Plataforma Integral de Gestión de Bienestar que automatice el ciclo de vida completo del alumno: desde la reserva autónoma hasta el registro de asistencia con descuento transaccional automático, dotando a la dirección de inteligencia de negocios en tiempo real.

## 2. Stack Tecnológico

Se eligió un stack moderno orientado a la escalabilidad y la experiencia de usuario mobile-first, alineado con los requerimientos del informe ejecutivo previo:

| Capa          | Tecnología         | Rol en el Sistema                                                    |
|---------------|--------------------|----------------------------------------------------------------------|
| Base de Datos | PostgreSQL 14+     | Fuente de verdad con integridad referencial y funciones PL/pgSQL     |
| Backend       | Node.js + Express  | API REST en arquitectura por capas (Rutas → Controladores → Modelos) |
| ORM / Queries | node-postgres (pg) | Pool de conexiones, transacciones y queries parametrizadas           |
| Autenticación | JWT + bcryptjs     | Tokens de sesión con diferenciación de roles admin/cliente           |
| Frontend      | React 18 + Vite    | SPA con enrutamiento por rol y comportamiento PWA                    |
| Estilos       | Tailwind CSS 3     | Sistema de diseño Stitch — tokens de color Sage Green + Terracotta   |
| Gráficos      | Recharts 2         | Visualizaciones de KPIs en el dashboard administrativo               |
| Deploy        | Render (Free)      | Prototipado con advertencia de spin-down en plan gratuito            |

## 3. Arquitectura del Sistema

### 3.1 Modelo de Datos PostgreSQL

Se implementó un esquema relacional con 6 tablas principales, restricciones de integridad referencial (ON DELETE CASCADE), índices estratégicos y dos funciones PL/pgSQL:

| Tabla             | Campos Clave                                                | Descripción                                                          |
|-------------------|-------------------------------------------------------------|----------------------------------------------------------------------|
| usuarios          | id, nombre, email, password_hash, rol                       | Almacena alumnos y admins. Rol CHECK: admin   cliente                |
| instructores      | id, nombre, especialidad                                    | 4 instructores del estudio                                           |
| paquetes          | id, nombre_plan, capacidad_clases, vigencia_dias, precio    | Planes: 10 clases, 20 clases, ilimitado (NULL = sin límite)          |
| paquetes_usuarios | usuario_id, paquete_id, clases_disponibles, estado          | Paquete asignado a cada alumno. Estado: activo   vencido   agotado   |
| clases            | nombre_clase, instructor_id, dia_semana, horario_inicio/fin | 20 clases semanales (0=Lun ... 6=Dom)                                |
| reservas          | usuario_id, clase_id, fecha_reserva, estado_asistencia      | Estado: pendiente   asistió   ausente. UNIQUE por alumno+clase+fecha |

**Funciones PL/pgSQL implementadas:**

- `descontar_clase(usuario_id)`: ejecuta el descuento transaccional dentro de la confirmación de asistencia
- `actualizar_paquetes_vencidos()`: marca paquetes vencidos por fecha, lista para ejecución periódica

**3.2 Arquitectura de Capas (Backend)****Flujo de una solicitud**

Cliente HTTP → `server.js` → `routes/*.routes.js`

→ `middleware/auth.js` (JWT + rol) → `middleware/businessRules.js` (reglas de negocio)

→ `controllers/*.controller.js` (lógica + queries SQL) → `config/db.js` (Pool PostgreSQL)

| Capa           | Archivo(s)                                              | Responsabilidad                                                                                               |
|----------------|---------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|
| Entrada        | <code>server.js</code>                                  | CORS, JSON parsing, montaje de rutas, error handler global                                                    |
| Rutas          | <code>auth / clases / reservas / admin.routes.js</code> | Definición de endpoints y cadena de middlewares                                                               |
| Autenticación  | <code>middleware/auth.js</code>                         | <code>verifyToken</code> , <code>requireAdmin</code> , <code>requireCliente</code>                            |
| Reglas Negocio | <code>middleware/businessRules.js</code>                | <code>validarSaldoActivo</code> , <code>validarVentanaReserva</code> , <code>validarVentanaCancelacion</code> |
| Controladores  | <code>*.controller.js</code>                            | Lógica de negocio, queries SQL parametrizadas, transacciones                                                  |
| BD             | <code>config/db.js</code>                               | Pool pg con reconexión automática                                                                             |

## 4. Reglas de Negocio Implementadas

---

Las tres reglas críticas se implementaron como middlewares encadenados en Express, garantizando que la validación ocurra antes de que cualquier controlador acceda a la base de datos:

### Regla 1 — Validación de Saldo Activo (`validarSaldoActivo`)

Antes de procesar cualquier reserva, el sistema verifica en la tabla `paquetes_usuarios` que el alumno tenga un paquete con estado "activo", con `fecha_vencimiento`  $\geq$  hoy y con `clases_disponibles`  $> 0$ . La única excepción es el plan ilimitado (`capacidad_clases` IS NULL), que nunca bloquea la reserva por saldo.

- HTTP 400 si paquete vencido: "Sin paquete activo. Adquiere o renueva tu plan."
- HTTP 400 si sin clases: "No tienes clases disponibles en tu paquete actual."

### Regla 2 — Cierre de Reservas a -15 Minutos (`validarVentanaReserva`)

Para clases programadas el mismo día de la semana que la solicitud, el middleware calcula la diferencia en minutos entre la hora actual y el `horario_inicio` de la clase. Si dicha diferencia es igual o menor a 15 minutos, la reserva (y también la cancelación, via `validarVentanaCancelacion`) queda bloqueada.

- HTTP 400: "Reserva cerrada. La clase inicia en menos de 15 minutos (N min)."
- Aplica también al cancelar: "No puedes cancelar con menos de 15 minutos de anticipación."

### Regla 3 — Descuento Transaccional Automatizado (`confirmarAsistencia`)

Cuando el administrador confirma que un alumno "asistió", el controlador ejecuta dentro de una transacción BEGIN/COMMIT la función PL/pgSQL `descontar_clase(usuario_id)`. Esta función resta 1 unidad de `clases_disponibles` y, si el saldo llega a 0, actualiza el estado del paquete a "agotado". En planes ilimitados, la función detecta `capacidad_clases` IS NULL y no descuenta.

- El descuento es atómico: si falla cualquier parte, se ejecuta ROLLBACK
- HTTP 400 si el alumno ya no tiene clases disponibles al momento de confirmar

## 5. Módulos del Frontend (React + Tailwind CSS)

### 5.1 Sistema de Diseño — Stitch Design System

El frontend sigue fielmente el sistema de diseño "RespiraApp Stitch", un lenguaje visual de Soft Minimalism con los siguientes tokens:

| Token                  | Valor                     | Uso                                                    |
|------------------------|---------------------------|--------------------------------------------------------|
| Primary (Sage Green)   | #4e635a                   | Botones principales, íconos activos, encabezados       |
| Secondary (Terracotta) | #83523b                   | Alertas, tags secundarios, barras de progreso críticas |
| Background             | #f2fbfd                   | Fondo general de la aplicación                         |
| Surface                | #ffffff                   | Cards y contenedores                                   |
| Primary Fixed          | #d1e8dd                   | Fondos de badges, banners informativos                 |
| Error                  | #ba1a1a                   | Estados de error, paquetes agotados                    |
| Display Font           | Playfair Display          | Títulos de sección y nombres de clases                 |
| Body Font              | Inter                     | Todo el texto funcional e interfaz                     |
| Íconos                 | Material Symbols Outlined | Iconografía consistente en toda la app                 |

### 5.2 Módulo Cliente (Alumno)

Interfaz mobile-first optimizada para la autonomía del alumno. Navegación mediante Bottom Bar con efecto glassmorphism. Incluye cuatro vistas:

| Vista         | Funcionalidad Principal                                                                                     | Reglas de Negocio Aplicadas                                              |
|---------------|-------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------|
| Inicio (Home) | Dashboard con próxima clase, resumen de paquete, estadísticas de asistencia y alerta de vencimiento próximo | Alerta automática si quedan ≤7 días de paquete                           |
| Clases        | Selector semanal de días, listado de clases del día con cupos en tiempo real, botones Reservar / Cancelar   | R1: bloqueo sin saldo · R2: bloqueo -15 min · Previene reserva duplicada |
| Mi Paquete    | Detalle completo del plan activo, barra de progreso de clases, fechas de inicio y vencimiento               | Muestra correctamente planes ilimitados (ícono ∞)                        |
| Historial     | Listado de todas las reservas con estado: Pendiente / Asistió / Ausente, con contadores de resumen          | Solo lectura — el alumno consulta pero no modifica estados               |

### 5.3 Módulo Administrador (Valentina)

Panel estratégico con Sidebar de navegación. Orientado a la inteligencia de negocios y el control operativo. Incluye tres vistas:

| Vista            | Funcionalidad Principal                                                                                                                                                                                         | Impacto Operativo                                                            |
|------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------|
| Dashboard (KPIs) | 4 tarjetas de métricas clave (alumnos activos, clases, reservas y asistencias hoy) + Gráfico de cartera (Activos / Por vencer / Dormidos) + ARPU mensual (line chart) + Heatmap de demanda por instructor y día | Elimina la "ceguera administrativa" — decisiones basadas en datos            |
| Asistencias      | Selector de día de semana, listado de clases con conteo de reservas, expandible para ver alumnos inscritos, botones "Asistió" / "Ausente" por alumno, opción de crear o eliminar clases                         | Confirmar asistencia en 2 clics — ejecuta descuento transaccional automático |
| Alumnos          | Listado de todos los clientes con buscador, estado y detalle del paquete activo (clases restantes, vencimiento), semáforo visual (verde/naranja/rojo), modal de asignación de paquetes                          | Reemplaza el proceso manual de "carga de planes" — asignación en 2 clics     |

## 6. Endpoints de la API REST

| Método | Ruta                               | Auth    | Descripción                                   |
|--------|------------------------------------|---------|-----------------------------------------------|
| POST   | /api/auth/login                    | Pública | Login → devuelve JWT + datos de usuario       |
| POST   | /api/auth/register                 | Pública | Registro de nuevo alumno                      |
| GET    | /api/auth/me                       | JWT     | Datos del usuario autenticado                 |
| GET    | /api/clases?dia=N                  | JWT     | Listado de clases (filtra por día 0-6)        |
| POST   | /api/clases                        | Admin   | Crear clase (valida instructor + horario)     |
| PUT    | /api/clases/:id                    | Admin   | Editar clase existente                        |
| DELETE | /api/clases/:id                    | Admin   | Eliminar clase y sus reservas en cascada      |
| GET    | /api/reservas/mis-reservas         | Cliente | Historial completo del alumno autenticado     |
| GET    | /api/reservas/mi-paquete           | Cliente | Paquete activo del alumno autenticado         |
| POST   | /api/reservas                      | Cliente | Crear reserva (aplica R1 + R2 + duplicado)    |
| DELETE | /api/reservas/:id                  | Cliente | Cancelar reserva (aplica R2 ventana -15min)   |
| GET    | /api/admin/kpis                    | Admin   | KPIs: cartera, demanda, ARPU, totales         |
| GET    | /api/admin/usuarios                | Admin   | Alumnos con paquete activo                    |
| POST   | /api/admin/usuarios/:id/paquete    | Admin   | Asignar paquete a alumno                      |
| GET    | /api/admin/clases/:id/reservas     | Admin   | Reservas de una clase en la fecha actual      |
| PATCH  | /api/admin/reservas/:id/asistencia | Admin   | Confirmar asistencia → ejecuta R3 (descuento) |

## 7. Estructura del Proyecto Entregado

### respira-profundo.zip — Organización de carpetas

db/  
  schema.sql → Tablas, índices, constraints, funciones PL/pgSQL, seed estático

backend/  
  server.js → Entrada + middlewares globales + error handler  
  seed.js → Genera hashes bcrypt reales e inserta datos de demo  
  fix-passwords.js → Repara contraseñas sin borrar datos (útil en setup)  
  .env.example → Variables de entorno documentadas  
  config/db.js → Pool de conexiones PostgreSQL  
  middleware/auth.js → JWT, requireAdmin, requireCliente  
  middleware/businessRules.js → Las 3 reglas de negocio como middlewares  
  routes/ → auth, clases, reservas, admin  
  controllers/ → auth, clases, reservas, admin

frontend/  
  vite.config.js → Configuración Vite + PWA (vite-plugin-pwa)



tailwind.config.js → Tokens del sistema de diseño Stitch  
src/context/AuthContext.jsx → JWT persistido en localStorage  
src/api/axios.js → Interceptores de token y 401  
src/pages/student/ → StudentHome, Classes, Package, History  
src/pages/admin/ → AdminDashboard, Classes, Students  
src/components/ → StudentLayout, AdminLayout, ui.jsx

## 8. Incidencias Detectadas y Resoluciones

Durante el proceso de desarrollo y pruebas se identificaron y resolvieron tres incidencias técnicas:

| #    | Incidencia                                                                                    | Causa Raíz                                                                                                                                                                                                        | Solución Aplicada                                                                                                                               |
|------|-----------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| I-01 | Usuarios demo no podían iniciar sesión                                                        | El script SQL original usaba un hash bcrypt de placeholder (\$2b\$10\$92lX...) que corresponde a la contraseña "password" y no a las credenciales documentadas                                                    | Se eliminaron los INSERT de usuarios del schema.sql y se creó seed.js que genera hashes correctos con bcrypt.hash() en tiempo de ejecución      |
| I-02 | Banner rojo "Sin clases disponibles" aparecía siempre aunque el alumno tuviera paquete activo | Discrepancia de nombre de campo: la API devuelve pu.estado pero el frontend consultaba paquete.estado_paquete (que es el alias usado solo en la consulta admin). El campo era siempre undefined → canBook = false | Corrección en StudentClasses.jsx, StudentHome.jsx y StudentPackage.jsx: paquete.estado_paquete → paquete.estado                                 |
| I-03 | Usuarios no existían en la BD tras ejecutar el schema.sql actualizado                         | El nuevo schema.sql (fix de I-01) ya no inserta usuarios, y seed.js no se había ejecutado. Los logs mostraban [LOGIN] Usuario no encontrado                                                                       | Se creó fix-passwords.js: script idempotente que crea los usuarios si no existen o actualiza sus hashes si ya existen, sin eliminar ningún dato |

## 9. Instrucciones de Instalación

### Prerrequisitos

- Node.js >= 18
- PostgreSQL >= 14 (con pgAdmin4 opcional para visualización)

### Paso 1 — Base de Datos

```
psql -U postgres -c "CREATE DATABASE respira_db;"
psql -U postgres -d respira_db -f db/schema.sql
```

### Paso 2 — Backend

```
cd backend && cp .env.example .env && npm install
node seed.js           # Crea usuarios con hashes reales
npm run dev            # http://localhost:4000
```

### Paso 3 — Frontend

```
cd frontend && npm install && npm run dev # http://localhost:5173
```

### **Credenciales de Demo**

Administrador: admin@respira.com / admin1234

Alumno 1: elena@mail.com / cliente123

Alumno 2: marco@mail.com / cliente123

Alumno 3: sofia@mail.com / cliente123

## 10. Beneficios Obtenidos y Riesgos Residuales

### 10.1 Beneficios Logrados

- **Eliminación total del proceso manual de reservas y descuento de clases**
- Autonomía del alumno: puede reservar, cancelar y consultar su saldo 24/7 sin intervención de Valentina
- Integridad transaccional garantizada: el descuento de clases ocurre en una transacción ACID
- Inteligencia de negocios: Valentina puede ver demanda por instructor, segmentación de cartera y ARPU
- Reducción de reclamos: los alumnos ven su saldo en tiempo real, eliminando disputas por cobros
- Escalabilidad: la arquitectura por capas permite agregar nuevos módulos sin refactoring mayor

### 10.2 Riesgos Residuales

- Plan Free de Render: latencia de 50+ segundos tras inactividad — migrar a plan pagado en producción
- Sin sistema de notificaciones push: las alertas de vencimiento son solo en-app, no proactivas por email o SMS
- Sin recuperación de contraseña: flujo de "olvidé mi contraseña" no implementado aún
- Curva de aprendizaje: los 4 instructores requieren capacitación para adoptar el sistema de asistencia
- Sin integración de pagos: la asignación de paquetes es manual desde el admin, no hay autoservicio de compra

## 11. Conclusión Ejecutiva

RespiraApp representa la transición exitosa del estudio "Respira Profundo" desde un modelo de gestión artesanal hacia una infraestructura digital robusta, implementada con un stack tecnológico moderno y probado en producción.

Las tres reglas de negocio críticas — validación de saldo, cierre de reservas a 15 minutos y descuento transaccional — fueron implementadas como middlewares encadenados en el backend, garantizando que ninguna operación del frontend pueda bypassar las validaciones de negocio. La base de datos en PostgreSQL actúa como fuente de verdad única e inalterable.

El módulo administrativo dota a Valentina de una herramienta de inteligencia de negocios que convierte el "caos manual" en ventaja competitiva: en dos clics puede pasar asistencia de una clase completa, ver qué instructor y horario llena más el salón, y detectar qué alumnos están por perder su paquete antes de que sea tarde.

#### Resumen de Entregables

- ✓ Script SQL completo (schema + funciones PL/pgSQL)

- ✓ API REST en Node.js/Express con 16 endpoints documentados
- ✓ 3 middlewares de reglas de negocio (R1, R2, R3)
- ✓ Frontend React PWA con módulos de alumno y administrador
- ✓ Sistema de diseño Stitch (Sage Green + Terracotta + Playfair Display)
- ✓ Dashboard analítico con Heatmap, Pie Chart y Line Chart (Recharts)
- ✓ Scripts de setup: seed.js + fix-passwords.js
- ✓ README completo con instrucciones de instalación paso a paso